ЛЕКЦИЯ 3. КОМПОНЕНТЫ ANDROID-ПРИЛОЖЕНИЙ

CONTEXT

- **Context** - объект, предоставляющий доступ к базовым функциям приложения
- Доступ к ресурсам, файловой системе, вызов активности и т.д.
- Activity является подклассом Context
- В коде: `MainActivity.this`
- Способы получения: `getApplicationContext()`, `getContext()`, `getBaseContext()`

CONTEXT: МЕТОДЫ

```
// Доступ к ресурсам
val color = ContextCompat.getColor(this, R.color.colorPrim
val string = getString(R.string.app_name)

// Запуск компонентов
startActivity(Intent(this, SecondActivity::class.java))

// Доступ к SharedPreferences
val prefs = getSharedPreferences("settings", Context.MODE_

// Доступ к файловой системе
val file = getFilesDir()
```

ACTIVITY

- **Activity** - компонент, представляющий один экран
- Объявляется в AndroidManifest.xml
- Имеет жизненный цикл (lifecycle)
- Каждая Activity - отдельный класс (наследник AppCompatActivity)

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

ЖИЗНЕННЫЙ ЦИКЛ ACTIVITY

ОСНОВНЫЕ

1. **onCreate**() - создание Activity
2. **onStart**() - видна, но не активна
3. **onResume**() - активна и видна
4. **onPause**() - частично видна
5. **onStop**() - полностью скрыта
6. **onDestroy**() - уничтожение
7. **onRestart**() - перезапуск

СЦЕНАРИИ

- Запуск: onCreate -> onStart -> onResume
- Кнопка "Назад": onPause -> onStop -> onDestroy
- Кнопка "Домой": onPause -> onStop
- Возврат: onRestart -> onStart -> onResume
- Входящий звонок: onPause -> onStop

ACTIVITY: ПРИМЕР LIFECYCLE

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Log.d("Lifecycle", "onCreate")

        // Восстановление состояния
        savedInstanceState?.let {
            val text = it.getString("KEY_TEXT")
            binding.textView.text = text
        }
    }

    override fun onResume() {
        super.onResume()
        Log.d("Lifecycle", "onResume")
        // Возобновление камеры, плеера и т.д.
    }
```

```
    override fun onPause() {
        super.onPause()
        Log.d("Lifecycle", "onPause")
        // Освобождение ресурсов
    }

    override fun onSaveInstanceState(outState: Bundle) {
        super.onSaveInstanceState(outState)
        outState.putString("KEY_TEXT", binding.textView.text.t
    }
}
```

ACTIVITY: ИЗМЕНЕНИЕ КОНФИГУРАЦИЙ

- Смена языка, ориентации, размера шрифта, темы
- По умолчанию Activity пересоздается
- Можно перехватить через `onConfigurationChanged()`

```
<activity
    android:name=".MainActivity"
    android:configChanges="orientation|screenSize|locale"
    android:label="@string/app_name" />
```

```
override fun onConfigurationChanged(newConfig: Configuration)
    super.onConfigurationChanged(newConfig)
```

```
    if (newConfig.orientation == Configuration.ORIENTATION_LAN
        // Горизонтальная ориентация
    }
}
```

⚠ Не рекомендуется: используйте ViewModel + onSaveInstanceState

ACTIVITY: TASK И BACK STACK

- **Task** - стек Activity, связанных с пользовательским действием
- Back Stack - LIFO структура (Last In, First Out)
- Каждый запуск new Intent добавляет Activity в стек
- Нажатие "Назад" извлекает текущую Activity

```
// Пример:
Activity A (запущена) -> Activity B (запущена) -> Activity
Back Stack: [A, B, C]

// Нажатие "Назад":
C уничтожается ->: [A, B]
```

```
B уничтожается ->: [A]
```

Флаги Intent: FLAG_ACTIVITY_NEW_TASK, FLAG_ACTIVITY_CLEAR_TOP, FLAG_ACTIVITY_SINGLE_TOP

ACTIVITY: TASK AFFINITY

- **Affinity** - идентификатор задачи, по умолчанию = имя пакета
- Позволяет группировать Activity в разные задачи
- Настраивается через `android:taskAffinity` в манифесте

```
<activity android:name=".ActivityA"
    android:taskAffinity="com.example.SHARED">

<activity android:name=".ActivityB"
    android:taskAffinity="com.example.PRIVATE">

// Разные affinity -> ActivityB откроется в новом стеке
```

ACTIVITY: РЕЖИМЫ ЗАПУСКА

STANDARD

- По умолчанию
- Создается новый экземпляр
- Каждый Intent -> новый Activity

SINGLETOP

- Если Activity уже наверху стека
- onNewIntent() вместо onCreate()
- Подходит для уведомлений

SINGLETASK

- Один экземпляр в стеке
- Если существует - brings to front
- Может иметь singleInstance affinity

SINGLEINSTANCE

- Отдельная задача (только один Activity)
- Не делит стек с другими Activity
- Подходит для звонков, карт

ACTIVITY: ЗАКРЫТИЕ ПРОЦЕССА

- Система может убить процесс при нехватке памяти
- Activity уничтожается, но состояние сохраняется в Bundle
- onSaveInstanceState() вызывается перед уничтожением
- onCreate() - восстанавливает состояние

```
// Сохранение
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putString("scroll_position", ...)
    outState.putParcelableArrayList("items", list)
    outState.putBoolean("is_loading", isLoading)
```

```
}

// Восстановление
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    savedInstanceState?.let {
        isLoading = it.getBoolean("is_loading", false)
    }
}

// ViewModel также сохраняет состояние
```

ФРАГМЕНТЫ (FRAGMENTS)

- **Fragment** - переиспользуемая часть UI внутри Activity
- Имеет собственный жизненный цикл
- Может использоваться в разных Activity
- Поддерживает back stack
- Два подхода: контейнер (replace) или отдельная разметка

ФРАГМЕНТЫ: ПРИМЕР

```
class FirstFragment : Fragment(R.layout.fragment_first) {

    private var _binding: FragmentFirstBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentFirstBinding.inflate(inflater, cont
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState:
        super.onViewCreated(view, savedInstanceState)
        binding.buttonNext.setOnClickListener {
            parentFragmentManager.beginTransaction()
                .replace(R.id.fragment_container, SecondFragme
                .addToBackStack(null)
```

```
                .commit()
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

FRAGMENTMANAGER: ТРАНЗАКЦИИ

```
supportFragmentManager.beginTransaction()
    .replace(R.id.container, fragment)  // замена фрагмент
    .add(R.id.container, fragment)      // добавление
    .remove(fragment)                   // удаление
    .hide(fragment)                     // скрытие
    .show(fragment)                     // показ
    .detach(fragment)                   // отсоединение
    .attach(fragment)                   // присоединение
    .addToBackStack(null)              // в back stack
    .commit()                           // выполнить
```

FRAGMENT RESULT API

- Передача данных между фрагментами без косвенных ссылок
- Заменяет старый подход requestKey/setTargetFragment
- Работает даже если фрагмент пересоздан

```
// Фрагмент-отправитель
val result = bundleOf("selected_item" to item)
setFragmentResult("requestKey", result)

// Фрагмент-получатель
setFragmentResultListener("requestKey") { key, bundle ->
    val item = bundle.getString("selected_item")
    // обработать результат
}
```

```
// Ключ - уникальная строка для пары listener + sender
```

DIALOGFRAGMENT

- Фрагмент, отображаемый как диалоговое окно
- Корректно обрабатывает lifecycle и configuration changes
- Можно отображать как обычный фрагмент

```
class ConfirmDialog : DialogFragment() {

    override fun onCreateDialog(savedInstanceState: Bundle?):
        return MaterialAlertDialogBuilder(requireContext())
            .setTitle("Подтверждение")
            .setMessage("Удалить элемент?")
            .setPositiveButton("Да") { _, _ ->
                setFragmentResult("confirm", bundleOf("deleted
            }
            .setNegativeButton("Отмена", null)
            .create()
```

```
    }

    override fun onStart() {
        super.onStart()
        // Настройка кнопок после show()
        (dialog as? AlertDialog)?.getButton(AlertDialog.BUTTON
            ?.setTextColor(Color.RED)
    }
}

// Показ
ConfirmDialog().show(supportFragmentManager, "confirm_dialog")
```

BOTTOMSHEETFRAGMENT

- Отображение контента снизу экрана
- Три состояния: Collapsed, Expanded, Hidden
- Рекомендуется `BottomSheetDialogFragment`

```
class MyBottomSheet : BottomSheetDialogFragment() {

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentBottomSheetBinding.inflate(inflater
        return binding.root
    }

    companion object {
        const val TAG = "MyBottomSheet"
    }
```

```
}

// Показ
MyBottomSheet().show(supportFragmentManager, MyBottomSheet.TAG

// Управление состоянием из Activity
bottomSheet.behavior.state = BottomSheetBehavior.STATE_EXPANDE
bottomSheet.behavior.isHideable = true
bottomSheet.behavior.peekHeight = 200
```

VIEWPAGER2 + FRAGMENTS

- Свайп между фрагментами (как в ViewPager)
- Поддерживает RTL, вертикальный режим
- Использует RecyclerView (эффективнее)

```
class ViewPagerAdapter(fragment: Fragment) : FragmentStateAdapter(fr

    override fun getItemCount(): Int = 3

    override fun createFragment(position: Int): Fragment {
        return when (position) {
            0 -> FirstTabFragment()
            1 -> SecondTabFragment()
            else -> ThirdTabFragment()
        }
    }
}

// В Activity/Fragment
```

```
val adapter = ViewPagerAdapter(this)
binding.viewPager.adapter = adapter
TabLayoutMediator(binding.tabLayout, binding.viewPager) { tab, pos
    tab.text = when(pos) {
        0 -> "Вкладка 1"
        1 -> "Вкладка 2"
        else -> "Вкладка 3"
    }
}.attach()
```

НАМЕРЕНИЯ (INTENTS)

- Механизм для описания операции: выбрать фото, отправить письмо, сделать звонок
- **Явные намерения**: указан класс Activity
- **Неявные намерения**: указан action/data/category
- Работают через Intent Filters в AndroidManifest

ЯВНЫЕ НАМЕРЕНИЯ

```
// Запуск Activity
val intent = Intent(this, SecondActivity::class.java)
intent.putExtra("KEY_DATA", "Hello")
startActivity(intent)

// Получение данных в SecondActivity
val data = intent.getStringExtra("KEY_DATA")

// Передача объекта
val user = User(1, "Иван")
intent.putExtra("USER", user)  // Parcelable
```

НЕЯВНЫЕ НАМЕРЕНИЯ

```
// Открытие браузера
val intent = Intent(Intent.ACTION_VIEW, Uri.parse("https:/
startActivity(intent)

// Отправка email
val intent = Intent(Intent.ACTION_SENDTO).apply {
    data = Uri.parse("mailto:")
    putExtra(Intent.EXTRA_EMAIL, arrayOf("test@mail.com"))
    putExtra(Intent.EXTRA_SUBJECT, "Тема")
    putExtra(Intent.EXTRA_TEXT, "Текст письма")
}
startActivity(intent)

// Разделение (share)
val intent = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, "Поделиться текстом")
```

```
}
startActivity(Intent.createChooser(intent, "Поделиться чер
```

PENDINGINTENT

- Обертка над Intent, дающая разрешение другому приложению выполнить действие от имени вашего
- Используется в: уведомления, AlarmManager, App Widgets
- Типы: getActivity, getBroadcast, getService, getForegroundService

```
// Уведомление с PendingIntent
val intent = Intent(this, DetailActivity::class.java)
val pendingIntent = PendingIntent.getActivity(
    this, 0, intent,
    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IM
)
```

```
val notification = NotificationCompat.Builder(this, CHANNEL_ID
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("Заголовок")
    .setContentText("Текст уведомления")
    .setContentIntent(pendingIntent)  // при нажатии
    .setAutoCancel(true)
    .build()

// Broadcast PendingIntent
val alarmIntent = PendingIntent.getBroadcast(
    this, 0, Intent(this, AlarmReceiver::class.java),
    PendingIntent.FLAG_IMMUTABLE
)
```

INTENT FILTERS: ПОДРОБНО

```
<activity android:name=".ShareActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.SEND" />
        <category android:name="android.intent.category.DEFAUL
        <data android:mimeType="text/plain" />
        <data android:mimeType="image/*" />
    </intent-filter>
</activity>
```

- **action** - тип операции (VIEW, SEND, EDIT)
- **category** - контекст запуска (DEFAULT, BROWSABLE)
- **data** - scheme + mimeType + path
- Приоритет: `android:priority` (от -1000 до 1000)
- API 31+: Intent PendingIntent.FLAG_IMMUTABLE/MUTABLE

DATA URIS

ФОРМАТ

```
scheme://authority/path?query#fragment

Примеры:
content://contacts/1
mailto:user@example.com
tel:+79001234567
file:///sdcard/photo.jpg
https://example.com/page?id=5
```

```
<data
    android:sche
    android:host
    android:path
    android:mime
/>

// Соответствие:
// https://examp
// https://examp
```

DEEP LINKS И APP LINKS

DEEP LINKS

- URI формата myapp://page/123
- Открывает приложение напрямую
- Может открывать chooser
- Нужно декларировать в манифесте

```
<intent-filter android:autoVerify="true">
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="https"
        android:host="mysite.com"
        android:pathPrefix="/app" />
</intent-filter>
```

- HTTPS UI
- Не показ
- Нужен D
- autoVerif

```
// Обработка
class DeepLin
    override
        super
        val u
        val p
        // на
    }
}
```

SHAREDPREFERENCES

- Простое хранение пар "ключ-значение"
- Данные хранятся в XML-файле
- Для простых настроек (не для больших объемов)

```
// Сохранение
val prefs = getSharedPreferences("settings", MODE_PRIVATE)
prefs.edit()
    .putString("username", "Иван")
    .putInt("age", 25)
    .putBoolean("dark_mode", true)
    .apply()  // асинхронно

// Чтение
val username = prefs.getString("username", "Гость")
```

ДИАЛОГОВЫЕ ОКНА

ТИПЫ ДИАЛОГОВ

DIALOGFRAGMENT

```
class MyDialog : DialogFragment() {
    override fun onCreateDialog(savedInstanceState: B
        return AlertDialog.Builder(requireContext())
            .setTitle("Заголовок")
            .setMessage("Текст")
            .setPositiveButton("OK") { _, _ -> }
            .setNegativeButton("Отмена") { _, _ -> }
            .create()
    }
}

// Показ
MyDialog().show(supportFragmentManager, "tag")
```

- **DialogFragment** - рекомендуемый подход
- **AlertDialog** - с кнопками, списком
- **DatePickerDialog** - выбор даты
- **TimePickerDialog** - выбор времени
- ProgressDialog - **устарел**

РЕСУРСЫ

- Ресурсы хранятся в каталоге `res/`
- Доступ через автоматически генерируемый класс `R`
- Типы: strings, colors, drawables, layouts, dimensions, menus, raw
- Ресурсы легче поддерживать и обновлять

СТРУКТУРА КАТАЛОГОВ РЕСУРСОВ

```
app/src/main/res/
├── drawable/           # Изображения
├── layout/             # Разметки XML
│   ├── activity_main.xml
│   └── fragment_first.xml
├── values/
│   ├── strings.xml     # Строки
│   ├── colors.xml      # Цвета
│   ├── dimens.xml      # Размеры
│   └── themes.xml      # Темы
├── values-night/       # Темная тема
├── mipmap/             # Иконки приложения
├── raw/                # Необработанные файлы
└── xml/                # XML-конфигурации
```

СТРОКОВЫЕ РЕСУРСЫ

```
<resources>
    <string name="app_name">Мое приложение</string>
    <string name="greeting">Привет, %s!</string>
    <string name="html_text"><b>Жирный</b> текст</string>
</resources>

// Использование в коде
val name = getString(R.string.greeting, "Иван")

// Использование в XML
// android:text="@string/app_name"
```

ЦВЕТОВЫЕ И РАЗМЕРНЫЕ РЕСУРСЫ

```
<resources>
    <color name="colorPrimary">#FF6200EE</color>
    <color name="colorPrimaryVariant">#FF3700B3</color>
    <color name="colorOnPrimary">#FFFFFFFF</color>
</resources>


<resources>
    <dimen name="margin_small">8dp</dimen>
    <dimen name="text_size_large">20sp</dimen>
</resources>

// Использование
val color = ContextCompat.getColor(this, R.color.colorPrim
```

```
binding.textView.setTextSize(TypedValue.COMPLEX_UNIT_SP, 2
```

ВИДЫ РАЗМЕТОК

FrameLayout
Наложение элементов

LinearLayout
Вертикально/ горизонтально

RelativeLayout
Относительное позиционирование

TableLayout
Таблица (строки/ колонки)

GridLayout
Сетка

ConstraintLayout
Рекомендуемый

С 2025 года: **Jetpack Compose** заменяет XML-разметки.

LINEARLAYOUT

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:weightSum="3">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:text="1/3"
        android:background="#FF0000" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="2"
        android:text="2/3"
        android:background="#00FF00" />
```

```
</LinearLayout>
```

CONSTRAINTLAYOUT

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello!"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Нажми"
        app:layout_constraintTop_toBottomOf="@id/textView"
        app:layout_constraintStart_toStartOf="parent"
```

```
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

CONSTRAINTLAYOUT: GUIDELINES

```
<androidx.constraintlayout.widget.ConstraintLayout ...>

    <androidx.constraintlayout.widget.Guideline
        android:id="@+id/guideline"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        app:layout_constraintGuide_percent="0.3" />

    <TextView
        android:id="@+id/leftView"
        .../
        app:layout_constraintEnd_toStartOf="@id/guideline" />

    <TextView
        android:id="@+id/rightView"
        .../
        app:layout_constraintStart_toStartOf="@id/guideline" /
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

- **Guideline** - невидимая линия для позиционирования
- Типы: vertical, horizontal
- От position: layout_constraintGuide_percent (0..1) или layout_constraintGuide_begin/end

CONSTRAINTLAYOUT: BARRIERS И CHAINS

BARRIERS

```
<androidx.constraintlayout.widget.Barrier
    android:id="@+id/barrier"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:barrierDirection="end"
    app:constraint_referenced_ids="viewA,viewB" />

<!-- viewC привязан к барьеру -->
<Button
    app:layout_constraintStart_toEndOf="@id/barrier" />
```

Динамическая граница по самому широкому/высокому view

```
<!-- spread chain -->
<Button app:layout_con
    app:layout_constra
<Button app:layout_con
    app:layout_constra

<!-- chain styles -->
app:layout_constraintH
app:layout_constraintH
app:layout_constraintH
```

Связывает views, р

MOTIONLAYOUT: ОСНОВЫ

- Расширение ConstraintLayout для анимации
- Описывает переходы между двумя состояниями (ConstraintSets)
- Файлы: motion_layout.xml + scene (transition definition)

```
<MotionLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layoutDescription="@xml/scene">

    <TextView
        android:id="@+id/animatedView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Двигай!" />
```

```
</MotionLayout>

// scene.xml
<Transition
    motion:constraintSetStart="@id/start"
    motion:constraintSetEnd="@id/end"
    motion:duration="1000">
    <OnSwipe />
</Transition>
```

АДАПТЕРЫ

- **Адаптер** - мост между данными и View-компонентами
- Преобразует данные в элементы для отображения
- Основные типы:
  - **ArrayAdapter** - простой список
  - **RecyclerView.Adapter** - рекомендуемый (эффективный)
  - **BaseAdapter** - кастомный

RECYCLERVIEW.ADAPTER

```
class ItemAdapter(private val items: List<Item>) :
    RecyclerView.Adapter<ItemAdapter.ViewHolder>() {

    class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val nameText: TextView = view.findViewById(R.id.nameText)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: In
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_layout, parent, false)
        return ViewHolder(view)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int
        holder.nameText.text = items[position].name
    }

    override fun getItemCount() = items.size
}
```

RecyclerView более эффективен, чем ListView/GridView (переиспользование View).

RECYCLERVIEW: VIEWHOLDER PATTERN

- **ViewHolder** - хранит ссылки на View элемента
- Создается один раз при onCreateViewHolder()
- Переиспользуется при scrollTo (не создаются новые View)
- ViewBinding вместо findViewById()

```
class ItemViewHolder(private val binding: ItemLayoutBinding) :
    RecyclerView.ViewHolder(binding.root) {

    fun bind(item: Item, onClick: (Item) -> Unit) {
        binding.nameText.text = item.name
        binding.root.setOnClickListener { onClick(item) }
    }
}

override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
    val binding = ItemLayoutBinding.inflate(
```

```
            LayoutInflater.from(parent.context), parent, false
        )
        return ItemViewHolder(binding)
    }

    override fun onBindViewHolder(holder: ItemViewHolder, position: Int
        holder.bind(items[position]) { onItemSelected(it) }
    }
```

RECYCLERVIEW: DIFFUTIL И LISTADAPTER

- **DiffUtil** - вычисляет разницу между двумя списками
- Минимальные изменения вместо notifyDataSetChanged()
- **ListAdapter** - обертка над DiffUtil

```
val DIFF_CALLBACK = object : DiffUtil.ItemCallback<Item>() {
    override fun areItemsTheSame(old: Item, new: Item) = old.id == new.id
    override fun areContentsTheSame(old: Item, new: Item) = old == new
}

class ItemListAdapter : ListAdapter<Item, ItemViewHolder>(DIFF_CALLBACK) {
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): Ite
        val binding = ItemLayoutBinding.inflate(LayoutInflater.from(parent
        return ItemViewHolder(binding)
```

```
    }

    override fun onBindViewHolder(holder: ItemViewHolder, position: Int) {
        holder.bind(getItem(position))
    }
}

// Обновление данных
adapter.submitList(newItems)  // асинхронный DiffUtil в фоне
```

RECYCLERVIEW: ITEMTOUCHHELPER

- Обработка свайпов и drag-and-drop
- Перетаскивание для reorder
- Свайп для удаления

```
val touchHelper = ItemTouchHelper(object : ItemTouchHelper.SimpleCallback(
    ItemTouchHelper.UP or ItemTouchHelper.DOWN,  // drag
    ItemTouchHelper.LEFT or ItemTouchHelper.RIGHT  // swipe
) {
    override fun onMove(rv: RecyclerView, vh: RecyclerView.ViewHolder,
        target: RecyclerView.ViewHolder): Boolean {
        val from = vh.adapterPosition
        val to = target.adapterPosition
        Collections.swap(items, from, to)
        adapter.notifyItemMoved(from, to)
        return true
    }

    override fun onSwiped(vh: RecyclerView.ViewHolder, direction: Int) {
        val pos = vh.adapterPosition
        items.removeAt(pos)
```

```
        adapter.notifyItemRemoved(pos)
    }

    override fun onChildDraw(c: Canvas, rv: RecyclerView, vh: ViewHolder,
        dX: Float, dY: Float, state: Int, isActive: Boolean) {
        // Кастомная отрисовка при свайпе
        val bg = ColorDrawable(Color.RED)
        bg.setBounds(right + dX.toInt(), top, right, bottom)
        bg.draw(c)
    }
})

touchHelper.attachToRecyclerView(recyclerView)
```

АНИМАЦИИ

ТИПЫ АНИМАЦИЙ

- **View Animation** - трансформации View, работает только с отрисовкой (трансформацией) View, не меняя реальных свойств объекта.
- **Property Animator** - анимация свойств
- **AnimatedVectorDrawable** - векторная
- **Layout Transition** - анимация изменений

СОВРЕМЕННЫЙ ПОДХОД

- **Jetpack Compose**: animate*AsState -
- **Shared Element Transition**
- **Navigation Transition**
- **MotionLayout**

PROPERTY ANIMATOR: ПРИМЕР

```
// ObjectAnimator
ObjectAnimator.ofFloat(button, "translationX", 0f, 200f).a
    duration = 1000
    start()
}

// ViewPropertyAnimator (conciliation syntax)
button.animate()
    .translationX(200f)
    .alpha(0.5f)
    .setDuration(1000)
    .start()

// ValueAnimator
ValueAnimator.ofFloat(0f, 1f).apply {
    duration = 2000
    addUpdateListener { animator ->
```

```
            button.alpha = animator.animatedValue as Float
        }
        start()
    }
```

WORKMANAGER: ФОНОВЫЕ ЗАДАЧИ

```
class SyncWorker(context: Context, params: WorkerParameters)
    : CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        return try {
            val data = apiService.syncData()
            database.save(data)
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }
}

// Запуск
val constraints = Constraints.Builder()
    .setRequiredNetworkType(NetworkType.CONNECTED)
    .build()
```

```
val workRequest = OneTimeWorkRequestBuilder<SyncWorker>()
    .setConstraints(constraints)
    .setBackoffCriteria(BackoffPolicy.EXPONENTIAL, 30, TimeUni
    .build()

WorkManager.getInstance(this).enqueue(workRequest)
```

WORKMANAGER VS SERVICE

WORKMANAGER

- Гарантирует выполнение даже после перезагрузки
- Сам выбирает оптимальное время
- Constraints (сеть, зарядка)
- Цепочки и параллельные задачи
- Для: синхронизация, резервное копирование

SERVICE

- Выполняется в foreground/background
- Непрерывная работа (музыка, загрузка)
- Прямой контроль lifecycle
- Foreground Service required для длительных задач (API 26+)
- Для: воспроизведение, отслеживание

Критерий: нужен ли непрерывный фон? Service. Иначе - WorkManager.

CONTENT PROVIDERS

- Механизм обмена данными между приложениями
- Стандартный CRUD интерфейс
- Использует URI для адресации
- Автоматическое управление конкурентным доступом

```
// URI: content://com.example.myapp/users/123
// scheme: content
// authority: com.example.myapp
// path: /users/123

class UsersProvider : ContentProvider() {

    companion object {
```

```
        const val AUTHORITY = "com.example.myapp"
        const val TABLE = "users"
        val CONTENT_URI = Uri.parse("content://$AUTHORITY/$TAB
    }

    override fun onCreate() = true

    override fun query(uri: Uri, projection: Array<String>?, s
        selectionArgs: Array<String>?, sortOrder: String?): Cu

    override fun insert(uri: Uri, values: ContentValues?): Uri
    override fun update(uri: Uri, values: ContentValues?, sel:
        selArgs: Array<String>?): Int { ... }
    override fun delete(uri: Uri, sel: String?, selArgs: Array
    override fun getType(uri: Uri): String? { ... }
}
```

CONTENTRESOLVER

- Клиентский интерфейс для доступа к ContentProvider
- Не нужно знать реализацию
- Работает через URI

```
// Чтение данных
val cursor = contentResolver.query(
    UsersProvider.CONTENT_URI,
    arrayOf("id", "name", "email"),
    null, null, null
)

cursor?.use {
    while (it.moveToNext()) {
        val id = it.getInt(it.getColumnIndexOrThrow("id"))
        val name = it.getString(it.getColumnIndexOrThrow("name
```

```
        val email = it.getString(it.getColumnIndexOrThrow("ema
    }
}

// Вставка
val values = ContentValues().apply {
    put("name", "Иван")
    put("email", "ivan@example.com")
}
val newUri = contentResolver.insert(UsersProvider.CONTENT_URI,

// Удаление
contentResolver.delete(newUri!!, null, null)
```

BROADCASTRECEIVER

- Получение системных или пользовательских broadcasts
- **Static**: объявлен в манифесте (API 26+ ограничения)
- **Dynamic**: регистрируется в runtime
- Для: изменение сети, зарядки, SMS, BOOT_COMPLETED

```
// Статическая регистрация (в манифесте)
<receiver android:name=".NetworkReceiver"
    android:exported="true">
    <intent-filter>
        <action android:name="android.net.conn.CONNECTIVITY_CH
    </intent-filter>
```

```
</receiver>

// Динамическая регистрация (в Activity/Fragment)
val receiver = NetworkReceiver()
val intentFilter = IntentFilter(ConnectivityManager.CONNECTIVI
registerReceiver(receiver, intentFilter)

// Важно: отмена регистрации
override fun onDestroy() {
    super.onDestroy()
    unregisterReceiver(receiver)
}
```

ПРИОРИТЕТ РАССЫЛОК

- Обрабатываются последовательно по приоритету
- Каждый получатель может: принять, изменить, отменить
- Передача данных через setResultData/setResultCode

```
// Отправка Ordered Broadcast
val intent = Intent("com.example.MY_ACTION")
sendOrderedBroadcast(intent, android.Manifest.permission.MY_PE

// Приемник с приоритетом
<receiver android:name=".FirstReceiver">
    <intent-filter android:priority="100">
        <action android:name="com.example.MY_ACTION" />
```

```
    </intent-filter>
</receiver>

class FirstReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        val data = getResultData()  // получение данных от пре
        abortBroadcast()  // остановить цепочку
        setResultData("modified")  // передать дальше
    }
}
```

SERVICES: STARTED VS BOUND

STARTED SERVICE

- Запускается через startService()
- Работает независимо от компонента
- Сам останавливается или stopSelf()
- Для: загрузка, обработка

```
class DownloadService : Service() {

    override fun onStartCommand(intent: Intent?,
        flags: Int, startId: Int): Int {
        // выполнить работу
        stopSelf(startId)
        return START_NOT_STICKY
    }

    override fun onBind(intent: Intent?)
        = null  // не привязан
}
```

BO

- Привязывается
- Предоставляет
- Живет пока есть
- Для: IPC, музыка

```
private val connectio
    override fun onSe
        service: IBin
        val binder = 
        myService = bi
    }
    override fun onSe
}

bindService(intent, c
// отвязка
unbindService(connect
```

FOREGROUND SERVICE

- Отображает уведомление (обязательно с API 26)
- Не убивается системой так легко
- Для длительных операций: загрузка файлов, GPS-трекинг

```
// Манифест (API 28+: permission)
<uses-permission android:name="android.permission.FOREGROUND_S

class LocationService : Service() {

    override fun onStartCommand(intent: Intent?, flags: Int, s
        val notification = NotificationCompat.Builder(this, "l
            .setContentTitle("Отслеживание")
            .setContentText("Определение местоположения...")
            .setSmallIcon(R.drawable.ic_location)
```

```
            .build()

        startForeground(1, notification)  //必须
        // запуск работы
        return START_STICKY
    }

    override fun onBind(intent: Intent?) = null
}
```

JETPACK COMPOSE: ОСНОВЫ

- Декларативный UI toolkit
- Вместо XML-разметок - Kotlin-функции
- `@Composable` аннотация
- Быстрое рендеринг: только изменившиеся композиции

```
@Composable
fun Greeting(name: String) {
    Text(
        text = "Hello, $name!",
        style = MaterialTheme.typography.headlineMedium,
        modifier = Modifier.padding(16.dp)
    )
}

@Composable
```

```
fun MyApp() {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        Greeting("Android")
        Greeting("Compose")
        Button(onClick = { /* действие */ }) {
            Text("Нажми меня")
        }
    }
}
```

COMPOSE: STATE, REMEMBER

- **remember** - сохранение состояния между перерисовками
- **mutableStateOf** - наблюдаемое состояние
- **mutableStateListOf** - наблюдаемый список

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }

    Column {
        Text("Счетчик: $count")
        Button(onClick = { count++ }) {
            Text("Добавить")
        }
    }
}
```

```
@Composable
fun ListExample() {
    val items = remember { mutableStateListOf<String>() }

    LazyColumn {
        items(items) { item ->
            Text(item)
        }
    }

    Button(onClick = { items.add("Item ${items.size}") }) {
        Text("Добавить")
    }
}
```

COMPOSE: LAUNCHEDEFFECT

- **LaunchedEffect(key)** - запуск корутины при изменении key
- Работает в scope композиции
- **SideEffect** - для не Compose кода
- **DisposableEffect** - cleanup при уходе

```
@Composable
fun Timer() {
    var seconds by remember { mutableStateOf(0) }

    LaunchedEffect(Unit) {
        while (true) {
            delay(1000)
            seconds++
        }
    }
```

```
    Text("Прошло: $seconds сек")
}

@Composable
fun LifecycleObserver() {
    val lifecycleOwner = LocalLifecycleOwner.current

    DisposableEffect(lifecycleOwner) {
        val observer = LifecycleEventObserver { _, event ->
            Log.d("Lifecycle", "Event: $event")
        }
        lifecycleOwner.lifecycle.addObserver(observer)
        onDispose {
            lifecycleOwner.lifecycle.removeObserver(observer)
        }
    }
}
```

NAVIGATION COMPONENT

- Управление навигацией между экранами
- Типобезопасная передача аргументов
- Интеграция с BottomNavigation, Drawer

```
// nav_graph.xml
<navigation
    android:id="@+id/nav_graph"
    app:startDestination="home">

    <fragment
        android:id="@+id/homeFragment"
        android:name="com.example.HomeFragment"
        android:label="Home">
        <action
            android:id="@+id/toDetail"
            app:destination="@id/detailFragment" />
    </fragment>
```

```
    <fragment
        android:id="@+id/detailFragment"
        android:name="com.example.DetailFragment">
        <argument
            android:name="itemId"
            app:argType="integer" />
    </fragment>
</navigation>

// B Activity
val navController = findNavController(R.id.nav_host)
NavigationUI.setupActionBarWithNavController(this, navController)

// Запуск навигации
navController.navigate(R.id.toDetail, bundleOf("itemId" to 42))
```

NAVIGATION: SAFE ARGS

- Генерирует типобезопасные классы для аргументов
- Компиляция: ошибка вместо runtime crash
- Два класса: Directions (для перехода) и Args (для получения)

```
// build.gradle
plugins {
    id("androidx.navigation.safeargs.kotlin")
}

// Использование Directions (отправитель)
class HomeFragment : Fragment() {
    override fun onViewCreated(view: View, savedInstanceState:
        binding.btnDetail.setOnClickListener {
            val action = HomeFragmentDirections
```

```
                .toDetail(itemId = 42, title = "Заголовок")
            findNavController().navigate(action)
        }
    }
}

// Использование Args (получатель)
class DetailFragment : Fragment() {
    override fun onViewCreated(view: View, savedInstanceState:
        val args = DetailFragmentArgs.fromBundle(requireArgume
        binding.titleText.text = args.title
        binding.idText.text = args.itemId.toString()
    }
}
```

РАЗРЕШЕНИЯ (PERMISSIONS)

NORMAL

- Предоставляются автоматически
- INTERNET, BLUETOOTH, ACCESS_WIFI_STATE
- Не требуют runtime запроса

DANGEROUS

- Требуют явного согласия пользователя
- CAMERA, READ_CONTACTS, LOCATION
- В group permissions

```
<uses-permission android:name="android.permission.CAMERA"
<uses-permission android:name="android.permission.READ_EXT

// Permission Groups
CAMERA  = CAMERA
LOCATION = ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION
CONTACTS = READ_CONTACTS, WRITE_CONTACTS, GET_ACCOUNTS
```

RUNTIME PERMISSIONS

```
// Проверка разрешения
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAM
    == PackageManager.PERMISSION_GRANTED) {
    // разрешено
}

// Запрос разрешения
ActivityCompat.requestPermissions(
    this,
    arrayOf(Manifest.permission.CAMERA),
    CAMERA_PERMISSION_CODE
)

// Обработка результата
override fun onRequestPermissionsResult(
    requestCode: Int, permissions: Array<String>, grantResults: Int
) {
    super.onRequestPermissionsResult(requestCode, permissions, gran
    when (requestCode) {
        CAMERA_PERMISSION_CODE -> {
```

```
            if (grantResults.isNotEmpty()
                && grantResults[0] == PackageManager.PERMISSION_GRA
                // разрешено
            } else {
                // показать объяснение
            }
        }
    }
}

// Современный подход (Activity Result API)
val launcher = registerForActivityResult(
    ActivityResultContracts.RequestPermission()
) { granted ->
    if (granted) { /* OK */ }
}
launcher.launch(Manifest.permission.CAMERA)
```

LIFECYCLE: LIFECYCLEOWNER И OBSERVER

- **LifecycleOwner** - компонент с lifecycle (Activity, Fragment)
- **LifecycleObserver** - наблюдатель за изменениями состояния
- **Lifecycle** - текущее состояние (CREATED, STARTED, RESUMED)

```
class MyLocationObserver(private val lifecycleOwner: LifecycleOwner

    @OnLifecycleEvent(Lifecycle.Event.ON_START)
    fun startLocationUpdates() {
        // включить GPS
    }
```

```
    @OnLifecycleEvent(Lifecycle.Event.ON_STOP)
    fun stopLocationUpdates() {
        // выключить GPS
    }
}

// Регистрация (устаревший подход)
lifecycle.addObserver(MyLocationObserver(this))

// Современный DefaultLifecycleObserver
class MyObserver : DefaultLifecycleObserver {
    override fun onStart(owner: LifecycleOwner) { /* ... */ }
    override fun onStop(owner: LifecycleOwner) { /* ... */ }
}

// В ViewModel - работает автоматически!
class MyViewModel : ViewModel() {
    init {
        viewModelScope.launch {
            lifecycle.repeatOnLifecycle(Lifecycle.State.STARTED) {
                // запуск при STARTED
            }
        }
    }
```

```
}
```

PROCESSLIFECYCLEOWNER

- Отслеживает lifecycle всего приложения (не компонента)
- Помогает определить: приложение на переднем/заднем плане
- Работает с задержкой (учитывает transition между Activity)

```
// Регистрация в Application.onCreate()
ProcessLifecycleOwner.get().lifecycle.addObserver(AppLifecycle

// Наблюдатель
class AppLifecycleObserver : DefaultLifecycleObserver {

    override fun onStart(owner: LifecycleOwner) {
        Log.d("App", "Приложение на переднем плане")
```

```
        // запуск обновлений, синхронизации
    }

    override fun onStop(owner: LifecycleOwner) {
        Log.d("App", "Приложение на заднем плане")
        // пауза, сохранение состояния
    }
}

// Текущее состояние
val isForeground = ProcessLifecycleOwner.get()
    .lifecycle.currentState.isAtLeast(Lifecycle.State.STARTED)
```

Задержка ~700ms позволяет корректно обрабатывать переходы между Activity

ИТОГИ: НАТИВНАЯ ANDROID-РАЗРАБОТКА

КОМПОНЕНТЫ

- Activity, Fragment
- Intent (явный/неявный)
- Service, BroadcastReceiver
- ContentProvider

СОВРЕМЕННЫЙ ПОДХОД

- Kotlin + Coroutines
- Jetpack Compose
- ViewModel + StateFlow
- Hilt/Dagger (DI)

Нативная разработка дает максимальную производительность и доступ ко всем API платформы.